

数值计算方法 第一次上机作业

自动化（控制）

1.1 问题描述

【1】级数 $S_N = \frac{1}{2^2-1} + \frac{1}{3^2-1} + \cdots + \frac{1}{N^2-1}$ ，其精确值为 $\frac{1}{2} \left(\frac{3}{2} - \frac{1}{N} - \frac{1}{N+1} \right)$ ，分别以单精度和双精度数据类型，按正向和反向两种顺序分别计算 $S_{10^2}, S_{10^4}, S_{10^6}$ ，指出各计算结果的有效位数，并分析有效位数不同的原因。

【2】分别以单精度和双精度数据类型，计算 $\sum_{i=1}^{10000} 0.1 - 1000$ 与 $\sum_{i=1}^{80000} 0.0125 - 1000$ ，并分析产生误差的原因。

第一题

2.1 问题分析

本题要求计算级数 $S_N = \sum_{k=2}^N \frac{1}{k^2-1}$ ，通过正向/反向求和与单/双精度的组合，观察计算结果的差异。

级数通项 $a_k = \frac{1}{k^2-1}$ 随 k 增大而迅速递减： $a_2 \approx 0.333$ ， $a_{10^6} \approx 10^{-12}$ ，两者相差约 10 个数量级。正向求和（ $k = 2$ 到 N ）时累加和逐渐增大到约 0.75，后续极小项可能因超出有效位数范围而被舍入丢弃（“大数吃小数”）；反向求和（ $k = N$ 到 2）则先累加小项，避免精度损失。单精度仅约 7 位有效数字，双精度约 15~16 位，因此上述效应在单精度下更为显著。

有效位数通过相对误差估算： $n \approx -\log_{10}(|S - S^*|/|S^*|)$ 。

2.2 程序代码

```
% 第一题
N_values = [1e2, 1e4, 1e6];
for idx = 1:length(N_values)
    N = N_values(idx);
    exact = 0.5 * (3/2 - 1/N - 1/(N+1)); % 精确值

    % 双精度正向/反向求和
```

```
S_df = 0; S_db = 0;
for k = 2:N,      S_df = S_df + 1/(k^2-1); end
for k = N:-1:2,  S_db = S_db + 1/(k^2-1); end

% 单精度正向/反向求和
S_sf = single(0); S_sb = single(0);
for k = 2:N,      S_sf = S_sf + single(1/(single(k)^2-1)); end
for k = N:-1:2,  S_sb = S_sb + single(1/(single(k)^2-1)); end

fprintf('N = %d\n', N);
fprintf('精确值:      %.15e\n', exact);
fprintf('双精度正向:   %.15e\n', S_df);
fprintf('双精度反向:   %.15e\n', S_db);
fprintf('单精度正向:   %.7e\n', S_sf);
fprintf('单精度反向:   %.7e\n\n', S_sb);
end
```

2.3 运行结果

```
>>Task1
N = 100
精确值:      7.400495049504950e-01
双精度正向:   7.400495049504949e-01
双精度反向:   7.400495049504949e-01
单精度正向:   7.4004948e-01
单精度反向:   7.4004954e-01

N = 10000
精确值:      7.499000049995000e-01
双精度正向:   7.499000049995057e-01
双精度反向:   7.499000049995000e-01
单精度正向:   7.4985212e-01
单精度反向:   7.4989998e-01

N = 1000000
精确值:      7.499990000005000e-01
双精度正向:   7.499990000005217e-01
双精度反向:   7.499990000004999e-01
单精度正向:   7.4985212e-01
单精度反向:   7.4999905e-01
```

2.4 结果分析

各计算结果的有效位数汇总如下：

N	双精度正向	双精度反向	单精度正向	单精度反向
10^2	15.8	15.8	7.5	7.3
10^4	14.1	16+	4.2	7.5
10^6	13.5	15.8	3.7	7.2

2.4.1 ”大数吃小数”现象

单精度正向求和在 $N = 10^4$ 和 $N = 10^6$ 时结果完全相同 (0.74985212)，说明从 $k = 10^4$ 到 $k = 10^6$ 新增的约 99 万项贡献完全为零。此时 $a_k < 10^{-8}$ ，而累加和约 0.75，单精度下 $0.75 + 10^{-8}$ 仍被舍入为 0.75，小项被完全”吞掉”。

2.4.2 反向求和的优势

单精度反向求和在 $N = 10^6$ 时仍保持约 7.2 位有效数字，远优于正向的 3.7 位。反向求和从极小项开始累加，小项间量级相近不会丢失精度，随后和值逐渐增大但加入的项也相应增大，量级差异始终可控。这验证了”连加时应按绝对值从小到大相加”的原则。

2.4.3 双精度也受求和顺序影响

双精度正向的有效位数从 15.8 位降至 13.5 位，而反向始终保持约 15~16 位。说明即使有高精度数据类型，不合理的算法设计仍会造成精度损失，两者需要配合。

2.4.4 N 增大的影响

N 越大，后续项越小、与累加和的量级差距越大、累加次数也越多，三重因素叠加使正向求和精度显著下降。单精度正向从 7.5 位 ($N = 10^2$) 降至 3.7 位 ($N = 10^6$)，损失近一半。

第二题

3.1 问题分析

本题计算两个理论值均为零的表达式： $E_1 = \sum_{i=1}^{10000} 0.1 - 1000$ ， $E_2 = \sum_{i=1}^{80000} 0.0125 - 1000$ 。

$0.1 = 1/10$ 和 $0.0125 = 1/80$ 的分母都含因子 5，无法被二进制精确表示。 0.1 在二进制下为无限循环小数 0.00011_2 ，存储时截断会引入表示误差。每次累加都带入该误差，经上万次累积后偏差可观。此外，表达式 2 累加 80000 次，为表达式 1 的 8 倍，误差累积更多。

3.2 程序代码

```
% 第二题
%% 表达式1: sum(0.1, 10000次) - 1000
S1_d = 0;          S1_s = single(0);
for i = 1:10000
    S1_d = S1_d + 0.1;
    S1_s = S1_s + single(0.1);
end
E1_d = S1_d - 1000;    E1_s = S1_s - single(1000);

%% 表达式2: sum(0.0125, 80000次) - 1000
S2_d = 0;          S2_s = single(0);
for i = 1:80000
    S2_d = S2_d + 0.0125;
    S2_s = S2_s + single(0.0125);
end
E2_d = S2_d - 1000;    E2_s = S2_s - single(1000);

%% 输出
fprintf('表达式1: sum(0.1, 10000次) - 1000\n');
fprintf('  双精度: %.15e\n  单精度: %.7e\n\n', E1_d, E1_s);
fprintf('表达式2: sum(0.0125, 80000次) - 1000\n');
fprintf('  双精度: %.15e\n  单精度: %.7e\n', E2_d, E2_s);
```

3.3 运行结果

```
>>Task2
表达式1: sum(0.1, 10000次) - 1000
  双精度: 1.588205122970976e-10
  单精度: -9.7106934e-02

表达式2: sum(0.0125, 80000次) - 1000
  双精度: 1.446551323169842e-09
  单精度: 6.8237305e-01
```

3.4 结果分析

结果汇总:

	双精度	单精度
$\sum 0.1 - 1000$	1.59×10^{-10}	-9.71×10^{-2}
$\sum 0.0125 - 1000$	1.45×10^{-9}	6.82×10^{-1}

3.4.1 二进制表示误差的累积

0.1 和 0.0125 在二进制下均为无限循环小数，截断后每次累加都引入微小误差。经过上万次叠加，误差从 10^{-8} （单精度单次舍入量级）累积到 10^{-2} 甚至 10^{-1} 量级。双精度单次舍入约 10^{-16} ，万次累加后仍仅 10^{-10} ，精度优势明显。

3.4.2 累加次数的影响

双精度下，表达式 2（80000 次）的误差 1.45×10^{-9} 约为表达式 1（10000 次） 1.59×10^{-10} 的 9 倍，累加次数增加 8 倍，误差增长约 9 倍，大致呈线性关系。单精度下表达式 2 的误差（0.68）也远大于表达式 1（-0.097），体现了累加次数对误差的放大效应。

3.4.3 大数吃小数的加剧

随着累加和增长到接近 1000，再加 0.0125 时两者量级差约 5 个数量级。单精度仅 7 位有效数字，0.0125 的低位信息在加法中被截断，每次加法都在损失精度。80000 次累加后总误差达到 0.68，偏差严重。双精度有 15~16 位有效数字，能充分保留小数信息，大数吃小数效应轻微。

心得体会

本次作业让我直观体会到了浮点运算中舍入误差的危害。第一题中单精度正向求和在 $N = 10^4$ 与 $N = 10^6$ 时结果完全相同，近百万项贡献被“吞掉”；而仅改变求和顺序就能将有效位数从 3.7 位恢复到 7.2 位。第二题则展示了二进制表示误差在大规模累加中的放大效应。这些实验加深了我对“大数吃小数”、求和顺序选择及数据精度等原则的理解。